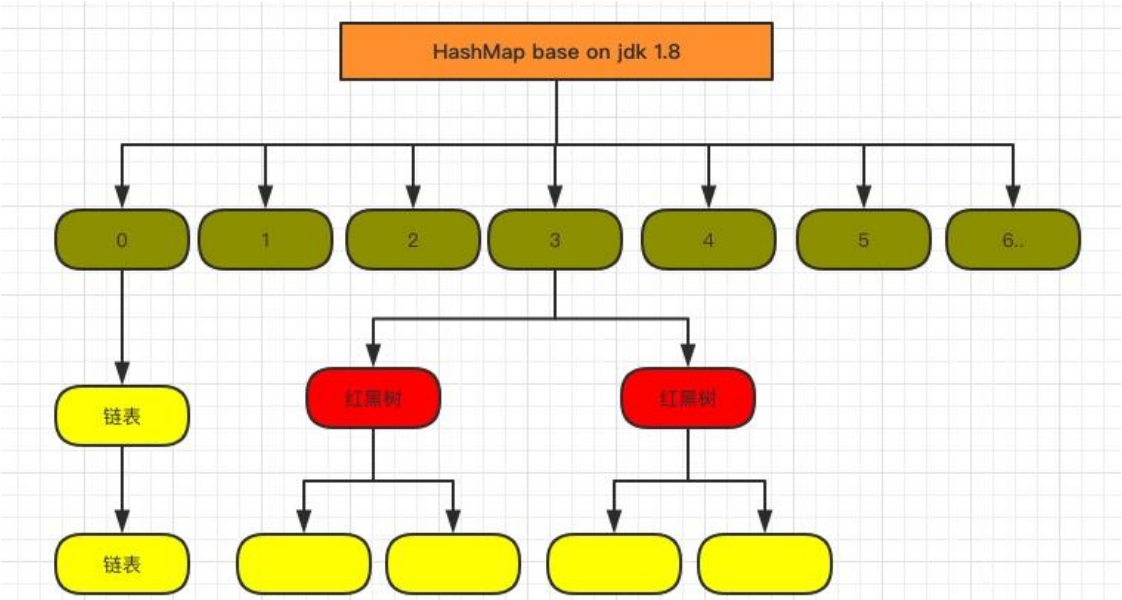
1 CHM 底层详解

1.1 数据结构

使用数组+链表+红黑树来实现，利用 CAS + synchronized 来保证并发更新的安全



1.2 源码分析

1.2.1 put 方法

```
 public V put(K key, V value) {
        return putVal(key, value, false);
    }

    final V putVal(K key, V value, boolean onlyIfAbsent) {
        //判断key 和value 是否有为空的情况，有则抛异常
        if (key == null || value == null) throw new NullPointerExceptio
n();
        //计算key 的hash 值，这里的hash 值不能为负
        int hash = spread(key.hashCode());
        //链表的深度
        int binCount = 0;
        //死循环
        for (Node<K,V>[] tab = table;;) {
            Node<K,V> f; int n, i, fh;
            //当hash 表为空，则初始化hash 表
            if (tab == null || (n = tab.length) == 0)
```

咕泡出品

```
                tab = initTable();
            //如果hash 表不为空，再判断下标i 中是否为空，如果满足，则直接创
建node 节点，放入到该下标
            else if ((f = tabAt(tab, i = (n - 1) & hash)) == null) {
                if (casTabAt(tab, i, null,
                             new Node<K,V>(hash, key, value, null)))
                    break;                   // no lock when adding to
empty bin
            }
            //如果Hash 表不为空，且下标i 中存在节点，则判断是否处于扩容，是
则协助扩容
            else if ((fh = f.hash) == MOVED)
                tab = helpTransfer(tab, f);
            //以上条件都不满足，则进行节点插入操作，可能是链表也可能是红黑树
            else {
                V oldVal = null;
                //对当前i 下标的node 节点进行上锁
                synchronized (f) {
                    if (tabAt(tab, i) == f) {
                        if (fh >= 0) {
                            binCount = 1;
                            //这一步是操作链表
                            for (Node<K,V> e = f;; ++binCount) {
                                K ek;
                                //当找到hash 值相等，且key 也相等的节点则
进行覆盖
                                if (e.hash == hash &&
                                    ((ek = e.key) == key ||
                                     (ek != null && key.equals(ek)))) {
                                    oldVal = e.val;
                                    if (!onlyIfAbsent)
                                        e.val = value;
                                    break;
                                }
                                //否则往链表最后插入一个新的节点
                                Node<K,V> pred = e;
                                if ((e = e.next) == null) {
                                    pred.next = new Node<K,V>(hash, key,
                                                              value, nu
ll);
                                    break;
                                }
                            }
                        }
                        //如果节点是树节点的话就操作红黑树
                        else if (f instanceof TreeBin) {
                            Node<K,V> p;
                            binCount = 2;
                            if ((p = ((TreeBin<K,V>)f).putTreeVal(hash,
```

咕泡出品必属精品

```
 key,
                                                                    value)) != n
ull) {
                                    oldVal = p.val;
                                    if (!onlyIfAbsent)
                                        p.val = value;
                                }
                            }
                        }
                    }
                    //在这里判断链表的深度是否大于等于8，如果满足条件则进入treeifyBin方法准备转成树
                    if (binCount != 0) {
                        if (binCount >= TREEIFY_THRESHOLD)
                            treeifyBin(tab, i);
                        if (oldVal != null)
                            return oldVal;
                        break;
                    }
                }
            }
            //增加chm的已存储的总数
            addCount(1L, binCount);
            return null;
        }

public V put(K key, V value) {
        return putVal(key, value, false);
    }
```

put 方法内部调用的是 putVal()方法，直接看 putVal()方法。

```
final V putVal(K key, V value, boolean onlyIfAbsent) {
        if (key == null || value == null) throw new NullPointerExceptio
n();
        int hash = spread(key.hashCode());
        //统计节点个数
        int binCount = 0;
```

首先判断 key 和 value 是否为空，如果为空的话直接抛出空指针异常。**注意：ConcurrentHashMap 的键和值不能为空。**

面试题：ConcurrentHashMap 的键值对为什么不能为 null，而 HashMap 却可以？

当你通过 get(k)获取对应的 value 时，如果获取到的是 null 时，你无法判断，它是 put（k,v）的时候 value 为 null，还是这个 key 从来没有做过映射。HashMap 是非并发的，可以通过 contains(key)来做这个判断。而支持并发的 Map 在调用 m.contains(key)和 m.get(key)的时候，m 可能已经不同了。

咸鱼出品 必属精品

接下来计算 hash 值，这里的 hash 值没有直接用 Object 类中的 hashCode()方法，而是经过了下面的变换：

```
static final int HASH_BITS = 0x7fffffff;
    static final int spread(int h) {
        return (h ^ (h >>> 16)) & HASH_BITS;
    }
```

这个原理和 HashMap 一样。

>>>的基本操作就是右移，然后高位补 0，这里的右移表示连符号位都要跟着左移；而>>只是右移数值位，不移动符号位。

这里是将原来的 hashcode 自身与右移 16 位之后进行异或运算，这样因为 hash 要和(length-1)进行与运算之后得到索引，(length-1)一般不会太大，所以 hash 的高位一般用不上。将原来 hashcode 的高 16 位和低 16 位做异或运算，这样新的 hash 也保留了高位的部分信息，会较少哈希冲突。

```
for (Node<K,V>[] tab = table;;) {
            Node<K,V> f; int n, i, fh;
            if (tab == null || (n = tab.length) == 0)
                tab = initTable();
            else if ((f = tabAt(tab, i = (n - 1) & hash)) == null) {
                if (casTabAt(tab, i, null,
                             new Node<K,V>(hash, key, value, null)))
                    break;                   // no lock when adding to
empty bin
            }
            else if ((fh = f.hash) == MOVED)
                tab = helpTransfer(tab, f);
            else {
```

声明一个 Node<K,V>类型的数组 tab，接下来有一个循环，循环内首先判断 tab 数组是否存在，如果不存在就初始化这个 tab，一会说何如初始化。

如果 tab 不为空，并且 计算出一个数组的下标 i = (n - 1) & hash，然后查看 tab 数组中的 i 位置是为 null。就直接 casTabAt(tab, i, null, new Node<K,V>(hash, key, value, null)) 使用 CAS 向 tab 中的第 i 个位置存入一个 node 类型的键值对然后退出循环。如果 casTabAt 失败了，后面还会再循环到这一步，也就是使用 CAS 自旋的往这个位置放入节点。

如果 tab 不为空，并且 `f.hash == MOVED`，f 就是前面 tab 中的第 i 个位置上的值，该值的哈希值如果==MOVED，MOVED 为常量-1，代表 ConcurrentHashMap 还有其他正在进行扩容。接下来执行 helpTransfer 就是帮助他进行扩容。

如果 tab 不为空，并且当前没有其他线程正在扩容：

```
V oldVal = null;
   synchronized (f) {
```

```
        if (tabAt(tab, i) == f) {
            if (fh >= 0) {
                binCount = 1;
                for (Node<K,V> e = f;; ++binCount) {
                    K ek;
                    if (e.hash == hash &&
                        ((ek = e.key) == key ||
                         (ek != null && key.equals(ek)))) {
                        oldVal = e.val;
                        if (!onlyIfAbsent)
                            e.val = value;
                        break;
                    }
                    Node<K,V> pred = e;
                    if ((e = e.next) == null) {
                        pred.next = new Node<K,V>(hash, key,
                                                  value, null);
                        break;
                    }
                }
            }
```

对链表(红黑树)中的根节点加了 synchronized 锁，那么在同一时间内，只能有一个线程对这条链表(红黑树)进行操作。拿到锁之后再判断一下根节点是否发生了变化，发生变化的话就要重新进入循环。没有发生变化的话, 接下来就是在链表中新增加一个节点。

fh 是根节点的哈希值，如果这个值大于等于 0 就代表是链表，否则就是红黑树。如果是链表的话，就循环的比较与当前链表中每个节点的 hash 值和 equals 是否相等，相等的话就覆盖，不相等的话就在链表的尾部插入新的节点。插入或覆盖结束之后代码继续往下运行。

```
else if (f instanceof TreeBin) {
                            Node<K,V> p;
                            binCount = 2;
                            if ((p = ((TreeBin<K,V>)f).putTreeVal(hash,
 key,
                                                                  value)) != n
ull) {
                                oldVal = p.val;
                                if (!onlyIfAbsent)
                                    p.val = value;
                            }
                        }
```

如果 f 是红黑树，那么就是已经树化了，数据的插入就是红黑树的插入。要注意的是，这里是红黑树的话 f 是一个 TreeBin 对象而不直接是红黑树的根节点，因为在红黑树的插入操作时有可能红黑树的根节点发生变化。如果对红黑树的根节点进行

咕泡出品 必属精品

加锁，put 之后根节点发生了变化，其他线程获得这个就可以获得新节点并加锁，这样就会出错。如果是 TreeBin 对象的话，锁住的就是这个对象，根节点发生变化不会应该加锁。

在 ConcurrentHashMap 中不是直接存储 TreeNode 来实现的，而是用 `TreeBin` 来包装 TreeNode 来实现的。也就是说在实际的 ConcurrentHashMap 桶中，存放的是 `TreeBin` 对象，而不是 TreeNode 对象。之所以 TreeNode 继承自 Node 是为了附带 next 指针，而这个 next 指针可以在 TreeBin 中寻找下一个 TreeNode，这里也是与 HashMap 之间比较大的区别。

```
if (binCount != 0) {
                                //static final int TREEIFY_THRESHOLD = 8;
                    if (binCount >= TREEIFY_THRESHOLD)
                        treeifyBin(tab, i);
                    if (oldVal != null)
                        return oldVal;
                    break;
                }
```

在数据插入之后，要判断链表的值是否大于等于 8，如果大于等于 8 就升级为红黑树。

1.2.1.1 treeifyBin 就是进行树化操作

```
private final void treeifyBin(Node<K,V>[] tab, int index) {
     Node<K,V> b; int n, sc;
     if (tab != null) {
         //在这里继续判断第二个条件，如果链表的深度大于等于8，且数组的长度大于等于64，则就转红黑树，否则尽心扩容
         if ((n = tab.length) < MIN_TREEIFY_CAPACITY)
             tryPresize(n << 1);
         //后面的逻辑就是链表转树
         else if ((b = tabAt(tab, index)) != null && b.hash >= 0) {
             synchronized (b) {
                 if (tabAt(tab, index) == b) {
                     TreeNode<K,V> hd = null, tl = null;
                     for (Node<K,V> e = b; e != null; e = e.next) {
                         TreeNode<K,V> p =
                             new TreeNode<K,V>(e.hash, e.key, e.val,
                                               null, null);
                         if ((p.prev = tl) == null)
                             hd = p;
                         else
                             tl.next = p;
                         tl = p;
                     }
                     setTabAt(tab, index, new TreeBin<K,V>(hd));
                 }
             }
```

咕泡讲品 必属精品

```
        }
    }
}
```

在进行树化操作时会先判断 table 数组的长度是否小于 MIN_TREEIFY_CAPACITY)，如果小于的话会使用 tryPresize 方法将容量扩大 2 倍。

1.2.1.2 initTable()

转而开一下 putVal()方法开头的 initTable() 方法：

```
private final Node<K,V>[] initTable() {
        Node<K,V>[] tab; int sc;
        while ((tab = table) == null || tab.length == 0) {
            if ((sc = sizeCtl) < 0)
                Thread.yield(); // 让出线程
                //正在初始化时将sizeCtl 设为-1
            else if (U.compareAndSwapInt(this, SIZECTL, sc, -1)) {
                try {
                    if ((tab = table) == null || tab.length == 0) {
                        int n = (sc > 0) ? sc : DEFAULT_CAPACITY;
                        @SuppressWarnings("unchecked")
                        Node<K,V>[] nt = (Node<K,V>[])new Node<?,?>[n];
                        table = tab = nt;
                        sc = n - (n >>> 2);
                    }
                } finally {
                    sizeCtl = sc; //扩容保护
                }
                break;
            }
        }
        return tab;
    }
```

initTable()用于里面 table 数组的初始化，值得一提的是 table 初始化是没有加锁的，那么如何处理并发呢？
由上面代码可以看到，当要初始化时会通过 CAS 操作将 sizeCtl 置为-1，而 sizeCtl 由 volatile 修饰，保证修改对后面线程可见。
这之后如果再有线程执行到此方法时检测到 sizeCtl 为负数，说明已经有线程在给扩容了，这个线程就会调用 Thread.yield()让出一次 CPU 执行时间。

1.2.1.3 对 put 方法的简单总结

①先传入一个 k 和 v 的键值对，不可为空（HashMap 是可以为空的），如果为空就直接报错。
②接着去判断 table 是否为空，如果为空就进入初始化阶段。
③如果判断数组中某个指定的桶是空的，那就直接把键值对插入到这个桶中作为

咸鱼出品 必属精品

头节点，而且这个操作不用加锁。
④如果这个要插入的桶中的 hash 值为-1，也就是 MOVED 状态（也就是这个节点是 forwordingNode），那就是说明有线程正在进行扩容操作，那么当前线程就进入协助扩容阶段。
⑤需要把数据插入到链表或者树中，如果这个节点是一个链表节点，那么就遍历这个链表，如果发现有相同的 key 值就更新 value 值，如果遍历完了都没有发现相同的 key 值，就需要在链表的尾部插入该数据。插入结束之后判断该链表节点个数是否大于 8，如果大于就需要把链表转化为红黑树存储。
⑥如果这个节点是一个红黑树节点，那就需要按照树的插入规则进行插入。
⑦put 结束之后，需要给 map 已存储的数量+1，在 addCount 方法中判断是否需要扩容

1.2.2 什么时候扩容

当往 hashMap 中成功插入一个 key/value 节点时，有两种情况可能触发扩容动作：
1、如果新增节点之后，所在链表的元素个数达到了阈值 8，则会调用 treeifyBin 方法把链表转换成红黑树，不过在结构转换之前，会对数组长度进行判断，实现如下：如果数组长度 n 小于阈值 MIN_TREEIFY_CAPACITY，默认是 64，则会调用 tryPresize 方法把数组长度扩大到原来的两倍，并触发 transfer 方法，重新调整节点的位置。

2、调用 put 方法新增节点时，在结尾会调用 addCount 方法记录元素个数，并检查是否需要进行扩容，当数组元素个数达到阈值时，会触发 transfer 方法，重新调整节点的位置。

扩容状态下其他线程对集合进行插入、修改、删除、合并、compute 等操作时遇到 ForwardingNode 节点会触发扩容 。

putAll 批量插入或者插入节点后发现存在链表长度达到 8 个或以上，但数组长度为 64 以下时会触发扩容。

注意：桶上链表长度达到 8 个或者以上，并且数组长度为 64 以下时只会触发扩容而不会将链表转为红黑树 。

1.2.3 如何扩容

1.2.3.1 addCount

在 putVal 方法的循环执行完毕之后，一个 Node 肯定放入进去了，此时就需要调用 addCount 方法来更新 baseCount 变量：

```
private final void addCount(long x, int check) {
    CounterCell[] as; long b, s;
      //counterCell 是用来存储总数的一个数组节点，默认长度是 2，baseCount 也是用来存储总数的
```

```
    //首先判断counterCell 是否为空，则表示当前不存在多线程在竞争，则尝试着去修改baseCount，如果失败则走if 里面的逻辑
    if ((as = counterCells) != null ||
        !U.compareAndSwapLong(this, BASECOUNT, b = baseCount, s = b + x)) {
        CounterCell a; long v; int m;
        boolean uncontended = true;
        //判断CounterCell 数组是否为空，是则调fullAddCount 方法
        if (as == null || (m = as.length - 1) < 0 ||
            (a = as[ThreadLocalRandom.getProbe() & m]) == null ||
            !(uncontended =
              U.compareAndSwapLong(a, CELLVALUE, v = a.value, v + x))) {
            fullAddCount(x, uncontended);
            return;
        }
        if (check <= 1)
            return;
        //将baseCount 和countCell 里面的数据全部相加
        s = sumCount();
    }
    if (check >= 0) {
        Node<K,V>[] tab, nt; int n, sc;
        while (s >= (long)(sc = sizeCtl) && (tab = table) != null &&
               (n = tab.length) < MAXIMUM_CAPACITY) {
            int rs = resizeStamp(n);
            if (sc < 0) {
                if ((sc >>> RESIZE_STAMP_SHIFT) != rs || sc == rs + 1 ||
                    sc == rs + MAX_RESIZERS || (nt = nextTable) == null ||
                    transferIndex <= 0)
                    break;
                if (U.compareAndSwapInt(this, SIZECTL, sc, sc + 1))
                    transfer(tab, nt);
            }
            else if (U.compareAndSwapInt(this, SIZECTL, sc,
                                         (rs << RESIZE_STAMP_SHIFT) + 2))
                transfer(tab, null);
            s = sumCount();
        }
    }
}
```

其中

```
private final void addCount(long x, int check) {
        CounterCell[] as; long b, s;
        if ((as = counterCells) != null ||
```

咕泡出品 必属精品

```
            !U.compareAndSwapLong(this, BASECOUNT, b = baseCount, s = b
 + x)) {
            CounterCell a; long v; int m;
            boolean uncontended = true;
            if (as == null || (m = as.length - 1) < 0 ||
                (a = as[ThreadLocalRandom.getProbe() & m]) == null ||
                !(uncontended =
                  U.compareAndSwapLong(a, CELLVALUE, v = a.value, v +
x))) {
                fullAddCount(x, uncontended);
                return;
            }
            if (check <= 1)
                return;
            //当前元素的数量
            s = sumCount();
        }
```

有一个名为 CounterCell 的数组，CounterCell 是一个简单的内部静态类，每个 CounterCell 都是一个用于记录元素个数的的单元。像一般的集合记录集合大小，直接定义一个 size 的成员变量即可，当出现改变的时候只要更新这个变量就行。为什么 ConcurrentHashMap 要用这种形式来处理呢？问题还是处在并发上，ConcurrentHashMap 是并发集合，如果用一个成员变量来统计元素个数的话，为了保证并发情况下共享变量的的安全，势必会需要通过加锁或者自旋来实现，如果竞争比较激烈的情况下，size 的设置上会出现比较大的冲突反而影响了性能，所以在 ConcurrentHashMap 采用了分片的方法来记录大小。

先判断 CounterCell 数组是否为空，不为空的话直接进入 if 代码执行，不再进行后面那个 CAS 操作，如果 CounterCell 为空的话就使用 CAS 的方式修改 baseCount，将其加上 x。如果 CAS 修改成功，上面代码也不会执行。如果 CAS 也失败，就进入 if 的代码块，对 CounterCell 数组进行初始化。

```
if (check >= 0) {
        Node<K,V>[] tab, nt; int n, sc;
        //s 是当前元素的个数
        while (s >= (long)(sc = sizeCtl) && (tab = table) != null &&
                (n = tab.length) < MAXIMUM_CAPACITY) {
            int rs = resizeStamp(n);
            //sc<0 表示已经有线程在进行扩容工作或者在进行初始化
            if (sc < 0) {

                //条件 1：检查是对容量 n 的扩容，保证 sizeCtl 与 n 是一块修改
好的
                //条件 2 与条件 3：进行 sc 的最小值或最大值判断。
                //条件 4 与条件 5：确保 tranfer()中的 nextTable 相关初始化逻
辑已走完。
               if ((sc >>> RESIZE_STAMP_SHIFT) != rs || sc == rs + 1 ||
```

```
                    sc == rs + MAX_RESIZERS || (nt = nextTable) == null
||
                    transferIndex <= 0)
                    break;
                    //有新线程参与扩容则sizeCtl 加1
                if (U.compareAndSwapInt(this, SIZECTL, sc, sc + 1))
                    transfer(tab, nt);
            }
            else if (U.compareAndSwapInt(this, SIZECTL, sc,
                                         (rs << RESIZE_STAMP_SHIFT) + 2))
                transfer(tab, null);
            s = sumCount();
        }
    }
```

上面这段代码是扩容的关键代码：

```
if (check >= 0) {
        Node<K,V>[] tab, nt; int n, sc;
        //s 是当前元素的个数
        while (s >= (long)(sc = sizeCtl) && (tab = table) != null &&
               (n = tab.length) < MAXIMUM_CAPACITY) {
```

if (check >= 0)判断 put 传入的 binCount(也就是 check)，check 为-1 的时候代表删除操作。通过 check 的正负判断是否进行扩容操作。
sizeCtl 为扩容的阈值。sizeCtl 默认的情况下等于 0，对 ConcurrentHashMap 进行初始化的时候会对 sizeCtl 减 1，初始化成功后将 sizeCtl 改为阈值（最大长度*0.75）。
s >= (long)(sc = sizeCtl)是进行判断当前的元素个数是否大于阈值，(tab = table) != null 数组的长度不为空，(n = tab.length) < MAXIMUM_CAPACITY 数组长度小于最大的容量

也就是当当前容量大于扩容阈值并且小于最大扩容值才扩容，如果 tab=null 说明正在初始化，死循环等待初始化完成。

1.2.3.2 resizeStamp()

```
private static int RESIZE_STAMP_BITS = 16;

private static final int RESIZE_STAMP_SHIFT = 32 - RESIZE_STAMP_BITS;

static final int resizeStamp(int n) {
        return Integer.numberOfLeadingZeros(n) | (1 << (RESIZE_STAMP_BI
TS - 1));
}
```

Integer.numberOfLeadingZeros(n)用于计算 n 转换成二进制后前面有几个 0。这个有什么作用呢？
首先 ConcurrentHashMap 的容量必定是 2 的幂次方，所以不同的容量 n 前面 0 的

咕泡出品必属精品

个数必然不同，这样可以保证是在原容量为 n 的情况下进行扩容。
(1 << (RESIZE_STAMP_BITS - 1)即是 1<<15，表示为二进制即是高 16 位为 0，低 16 位为 1：

```
Integer.numberOfLeadingZeros(16) 表示 16 的 2 进制最高位前面有多少个 0
00000000 00000000 00000000 00010000  前面有 27 个 0，值是 27，再把 27 转成 2 进制
00000000 00000000 00000000 00011011
00000000 00000000 10000000 00000000  1<<15
00000000 00000000 10000000 00011011
```

所以 resizeStamp(n)的返回值为：高 16 位置 0，第 16 位为 1，低 15 位存放当前容量 n，用于表示是对 n 的扩容。

rs 与 RESIZE_STAMP_SHIFT 配合可以求出新的 sizeCtl 的值，分情况如下：

1. sc < 0
   已经有线程在扩容，将 sizeCtl+1 并调用 transfer()让当前线程参与扩容。

2. sc >= 0
   表示没有线程在扩容，使用 CAS 将 sizeCtl 的值改为(rs << RESIZE_STAMP_SHIFT) + 2)。
   rs 即 resizeStamp(n)，记 temp=rs << RESIZE_STAMP_SHIFT。如当前容量为 8 时 rs 的值：

```
//rs
00000000 00000000 10000000 00011011
//temp = rs << RESIZE_STAMP_SHIFT，即 temp = rs << 16，左移16 后temp 最高位为1，所以temp 成了一个负数。
10000000 00011011 00000000 00000000
//sc = (rs << RESIZE_STAMP_SHIFT) + 2)
10000000 00011011 00000000 00000010
```

那么在扩容时 sizeCtl 值的意义：高 15 位为容量 n，低 16 位为并行扩容线程数+1

1.2.3.3 transfer()

jdk1.8 版本的 ConcurrentHashMap 支持并发扩容，transfer 方法是真正进行扩容的函数。

调用该扩容方法的地方有：

- java.util.concurrent.ConcurrentHashMap#addCount 向集合中插入新数据后更新容量计数时发现到达扩容阈值而触发的扩容
- java.util.concurrent.ConcurrentHashMap#helpTransfer 扩容状态下其他线程对集合进行插入、修改、删除、合并、compute 等操作时遇到 ForwardingNode 节点时触发的扩容

咕泡出品 必属精品

- java.util.concurrent.ConcurrentHashMap#tryPresize putAll 批量插入或者插入后发现链表长度达到 8 个或以上，但数组长度为 64 以下时触发的扩容

```
private final void transfer(Node<K,V>[] tab, Node<K,V>[] nextTab) {
        int n = tab.length, stride;  //stride 主要和CPU 相关
        if ((stride = (NCPU > 1) ? (n >>> 3) / NCPU : n) < MIN_TRANSFER
_STRIDE)  //每个线程处理桶的最小数目，可以看出核数越高步长越小，最小16 个。
            stride = MIN_TRANSFER_STRIDE; // subdivide range
        if (nextTab == null) {
            try {
                @SuppressWarnings("unchecked")
                Node<K,V>[] nt = (Node<K,V>[])new Node<?,?>[n << 1];  /
/扩容到2 倍
                nextTab = nt;
            } catch (Throwable ex) {      // try to cope with OOME
                sizeCtl = Integer.MAX_VALUE;  //扩容保护
                return;
            }
            nextTable = nextTab;
            transferIndex = n;  //扩容总进度，>=transferIndex 的桶都已分配
出去。
        }
        int nextn = nextTab.length;
          //扩容时的特殊节点，标明此节点正在进行迁移，扩容期间的元素查找要调
用其find()方法在nextTable 中查找元素。
        ForwardingNode<K,V> fwd = new ForwardingNode<K,V>(nextTab);
        //当前线程是否需要继续寻找下一个可处理的节点
        boolean advance = true;
        boolean finishing = false; //所有桶是否都已迁移完成。
        for (int i = 0, bound = 0;;) {
            Node<K,V> f; int fh;
            //此循环的作用是确定当前线程要迁移的桶的范围或通过更新i 的值确定
当前范围内下一个要处理的节点。
            while (advance) {
                int nextIndex, nextBound;
                if (--i >= bound || finishing)  //每次循环都检查结束条件
                    advance = false;
                //迁移总进度<=0，表示所有桶都已迁移完成。
                else if ((nextIndex = transferIndex) <= 0) {
                    i = -1;
                    advance = false;
                }
                else if (U.compareAndSwapInt
                         (this, TRANSFERINDEX, nextIndex,
                          nextBound = (nextIndex > stride ?
                                       nextIndex - stride : 0))) {  //t
ransferIndex 减去已分配出去的桶。
                    //确定当前线程每次分配的待迁移桶的范围为[bound, nextIn
```

咕泡出品必属精品

```
dex)
                    bound = nextBound;
                    i = nextIndex - 1;
                    advance = false;
                }
            }
            //当前线程自己的活已经做完或所有线程的活都已做完，第二与第三个条件应该是下面让"i = n"后，再次进入循环时要做的边界检查。
            if (i < 0 || i >= n || i + n >= nextn) {
                int sc;
                if (finishing) {  //所有线程已干完活，最后才走这里。
                    nextTable = null;
                    table = nextTab;  //替换新table
                    sizeCtl = (n << 1) - (n >>> 1); //调sizeCtl为新容量0.75倍。
                    return;
                }
                //当前线程已结束扩容，sizeCtl-1表示参与扩容线程数-1。
                if (U.compareAndSwapInt(this, SIZECTL, sc = sizeCtl, sc - 1)) {
                    //还记得addCount()处给sizeCtl赋的初值吗？相等时说明没有线程在参与扩容了，置finishing=advance=true，为保险让i=n再检查一次。
                    if ((sc - 2) != resizeStamp(n) << RESIZE_STAMP_SHIFT)
                        return;
                    finishing = advance = true;
                    i = n; // recheck before commit
                }
            }
            else if ((f = tabAt(tab, i)) == null)
                advance = casTabAt(tab, i, null, fwd);  //如果i处是ForwardingNode表示第i个桶已经有线程在负责迁移了。
            else if ((fh = f.hash) == MOVED)
                advance = true; // already processed
            else {
                synchronized (f) {  //桶内元素迁移需要加锁。
                    if (tabAt(tab, i) == f) {
                        Node<K,V> ln, hn;
                        //链表的处理
                        if (fh >= 0) {  //>=0表示是链表结点
                            //由于n是2的幂次方（所有二进制位中只有一个1），如n=16(0001 0000)，第4位为1，那么hash&n后的值第4位只能为0或1。所以可以根据hash&n的结果将所有结点分为两部分。
                            int runBit = fh & n;
                            Node<K,V> lastRun = f;
                            //找出最后一段完整的fh&n不变的链表，这样最后这一段链表就不用重新创建新结点了。
                            for (Node<K,V> p = f.next; p != null; p = p.
```

```
next) {
                                int b = p.hash & n;
                                if (b != runBit) {
                                    runBit = b;
                                    lastRun = p;
                                }
                            }
                            if (runBit == 0) {
                                ln = lastRun;
                                hn = null;
                            }
                            else {
                                hn = lastRun;
                                ln = null;
                            }
                            //lastRun 之前的结点因为fh&n 不确定，所以全部
需要重新迁移。

                            for (Node<K,V> p = f; p != lastRun; p = p.n
ext) {

                                int ph = p.hash; K pk = p.key; V pv = p.
val;

                                if ((ph & n) == 0)
                                    ln = new Node<K,V>(ph, pk, pv, ln);
                                else
                                    hn = new Node<K,V>(ph, pk, pv, hn);
                            }
                            //低位链表放在i 处
                            setTabAt(nextTab, i, ln);
                            //高位链表放在i+n 处
                            setTabAt(nextTab, i + n, hn);
                            setTabAt(tab, i, fwd);  //在原table 中设置F
orwardingNode 节点以提示该桶扩容完成。
                            advance = true;
                        }
                        //红黑树的处理
                        else if (f instanceof TreeBin) {
                            TreeBin<K,V> t = (TreeBin<K,V>)f;
                            TreeNode<K,V> lo = null, loTail = null;
                            TreeNode<K,V> hi = null, hiTail = null;
                            int lc = 0, hc = 0;
                            for (Node<K,V> e = t.first; e != null; e =
e.next) {

                                int h = e.hash;
                                TreeNode<K,V> p = new TreeNode<K,V>
                                    (h, e.key, e.val, null, null);
                                if ((h & n) == 0) {
                                    if ((p.prev = loTail) == null)
                                        lo = p;
                                    else
```

咕泡出品必属精品

```
                                    loTail.next = p;
                                loTail = p;
                                ++lc;
                            }
                            else {
                                if ((p.prev = hiTail) == null)
                                    hi = p;
                                else
                                    hiTail.next = p;
                                hiTail = p;
                                ++hc;
                            }
                        }
                        //如果拆分后的树的节点数量已经少于6个就需要重
新转化为链表
                        ln = (lc <= UNTREEIFY_THRESHOLD) ? untreeif
y(lo) :
                            (hc != 0) ? new TreeBin<K,V>(lo) : t;
                        hn = (hc <= UNTREEIFY_THRESHOLD) ? untreeif
y(hi) :
                            (lc != 0) ? new TreeBin<K,V>(hi) : t;
                            //CAS 存储在nextTable 的i 位置上
                        setTabAt(nextTab, i, ln);
                          //CAS 存储在nextTable 的i+n 位置上
                        setTabAt(nextTab, i + n, hn);
                        //CAS 在原table 的i 处设置forwordingNode 节点，
表示这个这个节点已经处理完毕
                        setTabAt(tab, i, fwd);
                        advance = true;
                    }
                }
            }
        }
    }
}
```

看第一部分：

```
private final void transfer(Node<K,V>[] tab, Node<K,V>[] nextTab) {
        int n = tab.length, stride;  //stride 主要和CPU 相关
        if ((stride = (NCPU > 1) ? (n >>> 3) / NCPU : n) < MIN_TRANSFER
_STRIDE)  //每个线程处理桶的最小数目，可以看出核数越高步长越小，最小步长16 个。
            stride = MIN_TRANSFER_STRIDE; // subdivide range
        if (nextTab == null) {
            try {
                @SuppressWarnings("unchecked")
                Node<K,V>[] nt = (Node<K,V>[])new Node<?,?>[n << 1];  /
/扩容到2 倍
                nextTab = nt;
```

咕泡出品 必属精品

```
            } catch (Throwable ex) {      // try to cope with OOME
                sizeCtl = Integer.MAX_VALUE;  //扩容保护
                return;
            }
            nextTable = nextTab;
            transferIndex = n;  //扩容总进度，>=transferIndex 的桶都已分配
出去。
            int nextn = nextTab.length;
        }
```

n 为扩容之前数组的长度，stride 主要和 CPU 相关，含义为步长，每个线程在扩容时拿到的长度，最小为 16。

nextTab 为新的 table，如果 nextTab 为空就新建一个 table，大小为原来的 2 倍，代表双倍扩容。

transferIndex=n； n 为扩容前的大小。

int nextn = nextTab.length; nextn 为扩容后数组的大小

```
//构造一个 ForwardingNode 用于多线程之间的共同扩容情况
 ForwardingNode<K,V> fwd = new ForwardingNode<K,V>(nextTab);
 boolean advance = true; //继续遍历的确认标志
 boolean finishing = false; //所有桶是否都已迁移完成标志
```

fwd 是一个标志，标明此节点正在进行迁移。当其他线程进行操作的时候发现这个位置存放的是 fwd 就知道正在进行扩容。、

advance 是遍历的确认标志，是否再往前进行遍历。 finishing 是所有桶是否都已迁移完成标志。

```
//遍历每个节点
  for (int i = 0, bound = 0;;) {
      Node<K,V> f; int fh; //定义一个节点和一个节点状态判断标志 fh
      while (advance) {
          int nextIndex, nextBound;
          if (--i >= bound || finishing) 每次循环都检查结束条件
              advance = false;
          //迁移总进度<=0，表示所有桶都已迁移完成
          else if ((nextIndex = transferIndex) <= 0) {
              i = -1;
              advance = false;
          }
```

接下来开始遍历每一个节点，bound 是当前步长结尾的位置。初始化一个节点 f 和一个节点状态判断标志 fh

while 循环的作用是确定当前线程要迁移的桶的范围或通过更新 i 的值确定当前范围内下一个要处理的节点。

哈尔出品 必属精品

当前步长内元素转移完成后 i = -1

```
else if (U.compareAndSwapInt
          (this, TRANSFERINDEX, nextIndex,
           nextBound = (nextIndex > stride ?
                        nextIndex - stride : 0))) {  //transferIndex 减去已分配出去的桶。
     //确定当前线程每次分配的待迁移桶的范围为[bound, nextIndex)
     bound = nextBound;
     i = nextIndex - 1;
     advance = false;
 }
```

这里是一个 CAS 的计算来修改 TRANSFERINDEX(转移到的下标)，配合上面一段代码计算出当前线程操作数组的具体区域。

```
if (i < 0 || i >= n || i + n >= nextn) {
        int sc;
        //如果原table 已经复制结束
        if (finishing) {
            nextTable = null; //可以看出在扩容的时候nextTable 只是类似于一个temp 用完会丢掉
            table = nextTab;
            sizeCtl = (n << 1) - (n >>> 1); //修改扩容后的阀值，应该是现在容量的0.75 倍
            return;//结束循环
        }
        //采用CAS 算法更新SizeCtl。
        if (U.compareAndSwapInt(this, SIZECTL, sc = sizeCtl, sc - 1)) {
            if ((sc - 2) != resizeStamp(n) << RESIZE_STAMP_SHIFT)
                return;
            finishing = advance = true;
            i = n; // recheck before commit
        }
    }
```

i < 0 || i >= n || i + n >= nextn 一共三个条件。1.当前线程自己的活已经做完或所有线程的活都已做完，第二与第三个条件应该是下面让"i = n"后，再次进入循环时要做的边界检查。

如果当前线程的工作做完后发现此时已经 finialing 了，就可以把 sizeCtl 改为新的值结束循环。

```
 //CAS 算法获取数组第i 的节点，为空就设为forwordingNode
     else if ((f = tabAt(tab, i)) == null)
         advance = casTabAt(tab, i, null, fwd);
    //如果这个节点的hash 值是MOVED，就表示这个节点是forwordingNode 节点，就表示这个节点已经被处理过了，直接跳过
```

咕泡出品 必屬精品

```
    else if ((fh = f.hash) == MOVED)
        advance = true; // already processed
```

如注释所示，CAS 算法获取数组第 i 个位置的值，为空就设为 fwd 标志位。如果这个节点的 hash 值是 MOVED，代表内容已经是 fwd，被其他线程处理过了，直接跳过，继续前进。

```
//处理链表
if (tabAt(tab, i) == f) {//再做一次校验

    //一个高位节点,一个低位节点
   Node<K, V> ln, hn;//ln 表示低位，hn 表示高位；接下来这段代码的作用是把链表拆分成两部分，0 在低位，1 在高位

        */为什么要分成高位和低位两种节点？
        1.如果 一个一个节点迁移，需要计算很多次节点的 hash 值。有了高低位链表，只需要在这里进行一次计算，且迁移也是一次（lastRun）
        2.对于一个节点迁移到新的数组中，其对应的位置只有两种可能，一种是下标不变，另一种是原始下标+数组的长度，所以这里把高位链添加到 i+n 的地方
                下次获取的时候相当于一下就获取到了低位链和高位链的数据
                */

        if (fh >= 0) {  //当前节点的哈希值

            /*由于n 是2 的幂次方（所有二进制位中只有一个1），如n=16(0001 0000)，第4 位为1，那么 hash&n 后的值第4 位只能为0 或1。
            所以可以根据hash&n 的结果将所有结点分为两部分。就是尾结点的第x 位等于0？ 0->低位   否则->高位
            低位节点哈希值的第x 位等于0(fh & n)
            高位节点哈希值的第x 位不等于0(fh & n) */
            int runBit = fh & n;          //fh 是当前节点的哈希值，n 是tab 长度，runBit 是fh 的第n 位的值(0 或1)

            //lastRun 是最终要处理的节点，这里先让指向根节点，后面还会寻找真正的lastRun
            Node<K, V> lastRun = f;

            //一条链表上的hash 值相等吗？   不一定相等，下标相等只是 (n-1)&hash 相等。

            //这个for 循环找到lastRun，从 lastRun 开始后面的要在低位都在低位，要在高位都在高位
            for (Node<K, V> p = f.next; p != null; p = p.next) {
                // 取于桶中每个节点的 hash 值
                //下一个节点的hash&n   当前节点的hash&n，因为一直循环所以就是尾结点和头结点的hash&n 比较
```

```
                int b = p.hash & n;
                // 如果节点的 hash 值和首节点的 hash 值取于结果不同
                if (b != runBit) {
                    runBit = b;// 更新 runBit 为尾结点的，用于下面判断 lastRun 该赋值给 ln 还是 hn。
                    lastRun = p;// 这个 lastRun 保证后面的节点与自己的取于值相同，避免后面没有必要的循环，因为上面是从 p 开始循环的。
                    // 所以这里到lastrun 就不循环了
                }
            }

            if (runBit == 0) {//如果最后更新的 runBit，设置低位节点
                ln = lastRun;
                hn = null;
            } else {//否则，设置高位节点
                hn = lastRun;// 如果最后更新的 runBit 是 1，设置高位节点
                ln = null;
            }
            //构造高位以及低位的链表
            // 再次循环，生成两个链表，lastRun 作为停止条件，这样就是避免无谓的循环（lastRun 后面都是相同的取于结果）
              // 将原本的一个链表根据 hash&n 分为 2 个链表，构建新链表采用头插法
            // 无法概括两个新链表相对旧链表的顺序，有很多可能，并不是一个正序，一个倒序

            // 这个 for 循环，把 lastRun 前面装配到高位节点或者低位节点
            for (Node<K, V> p = f; p != lastRun; p = p.next) {
                int ph = p.hash;
                K pk = p.key;
                V pv = p.val;
                // 如果与运算结果是 0，那么就还在低位
                if ((ph & n) == 0)// 如果是 0 ，那么创建低位节点
                    ln = new Node<K, V>(ph, pk, pv, ln);
                else // 1 则创建高位
                    hn = new Node<K, V>(ph, pk, pv, hn);
            }

            setTabAt(nextTab, i, ln);//将低位的链表放在 i 位置也就是不动 低位链不需要变
            setTabAt(nextTab, i + n, hn);//将高位链表放在 i+n 位置,n 是数组的长度,是假如当前 14 14+16=30
            setTabAt(tab, i, fwd);//把旧 table 的 hash 桶中放置转发节点，表明此 hash 桶已经被处理
            advance = true;
}
```

咕泡出品必属精品

这里先根据 fh 判断头的位置是链表的头节点还是树的根节点，如果是链表的话就执行链表转移，在转移过程中使用的是 CAS 算法。使用头插法进行转移。

如果是红黑树的话转移的方法和 HashMap1.8 中对红黑树的转移是一样的，是使用了两个链表，一个是高位链表一个是低位链表：

```
else if (f instanceof TreeBin) {
      TreeBin<K,V> t = (TreeBin<K,V>)f;
      //lo 为低位链表头结点，loTail 为低位链表尾结点，hi 和 hiTail 为高位链表头尾结点
      TreeNode<K,V> lo = null, loTail = null;
      TreeNode<K,V> hi = null, hiTail = null;
      int lc = 0, hc = 0;
      //同样也是使用高位和低位两条链表进行迁移
      //使用 for 循环以链表方式遍历整棵红黑树，使用尾插法拼接 ln 和 hn 链表
      for (Node<K,V> e = t.first; e != null; e = e.next) {
          int h = e.hash;
          //这里面形成的是以 TreeNode 为节点的链表
          TreeNode<K,V> p = new TreeNode<K,V>
              (h, e.key, e.val, null, null);
          if ((h & n) == 0) {
              if ((p.prev = loTail) == null)
                  lo = p;
              else
                  loTail.next = p;
              loTail = p;
              ++lc;
          }
          else {
              if ((p.prev = hiTail) == null)
                  hi = p;
              else
                  hiTail.next = p;
              hiTail = p;
              ++hc;
          }
      }
      //形成中间链表后会先判断是否需要转换为红黑树:
      //1、如果符合条件则直接将 TreeNode 链表转为红黑树，再设置到新数组中去
      //2、如果不符合条件则将 TreeNode 转换为普通的 Node 节点，再将该普通链表设置到新数组中去
      //(hc != 0) ? new TreeBin<K,V>(lo) : t 这行代码的用意在于，如果原来的红黑树没有被拆分成两份，那么迁移后它依旧是红黑树，可以直接使用原来的 TreeBin 对象
      ln = (lc <= UNTREEIFY_THRESHOLD) ? untreeify(lo) :
      (hc != 0) ? new TreeBin<K,V>(lo) : t;
      hn = (hc <= UNTREEIFY_THRESHOLD) ? untreeify(hi) :
      (lc != 0) ? new TreeBin<K,V>(hi) : t;
```

咕泡出品 必属精品

```
        //setTabAt 方法调用的是 Unsafe 类的 putObjectVolatile 方法
        //使用 volatile 方式的 putObjectVolatile 方法，能够将数据直接更新回
主内存，并使得其他线程工作内存的对应变量失效，达到各线程数据及时同步的效果
        //使用 volatile 的方式将 ln 链设置到新数组下标为 i 的位置上
        setTabAt(nextTab, i, ln);

        //使用 volatile 的方式将 hn 链设置到新数组下标为 i + n(n 为原数组长度)
 的位置上
        setTabAt(nextTab, i + n, hn);

        //迁移完成后使用 volatile 的方式将占位对象设置到该 hash 桶上，该占位对
象的用途是标识该hash 桶已被处理过，以及查询请求的转发作用
        setTabAt(tab, i, fwd);

        //advance 设置为 true 表示当前 hash 桶已处理完，可以继续处理下一个 ha
sh 桶
        advance = true;
    }
```

1.2.3.4 helpTransfer()

这个方法是帮助其他线程进行扩容。添加、删除节点之处都会检测到 table 的第 i 个桶是 ForwardingNode 的话会调用 helpTransfer()方法。

```
final Node<K,V>[] helpTransfer(Node<K,V>[] tab, Node<K,V> f) {
        Node<K,V>[] nextTab; int sc;
        if (tab != null && (f instanceof ForwardingNode) &&
            (nextTab = ((ForwardingNode<K,V>)f).nextTable) != null) {
            int rs = resizeStamp(tab.length);
            while (nextTab == nextTable && table == tab &&
                   (sc = sizeCtl) < 0) {
                if ((sc >>> RESIZE_STAMP_SHIFT) != rs || sc == rs + 1 |
|
                    sc == rs + MAX_RESIZERS || transferIndex <= 0)
                    break;
                if (U.compareAndSwapInt(this, SIZECTL, sc, sc + 1)) {
                    transfer(tab, nextTab);
                    break;
                }
            }
            return nextTab;
        }
        return table;
    }
```

咕泡出品 必属精品

1.2.3.5 tryPresize()

putAll 批量插入或者插入节点后发现链表长度达到 8 个或以上，但数组长度为 64 以下时触发的扩容会调用到这个方法.

```
private final void tryPresize(int size) {
        //根据传入的size 计算出真正的新容量，因为新容量需要是2 的幂次方。
        int c = (size >= (MAXIMUM_CAPACITY >>> 1)) ? MAXIMUM_CAPACITY :
            tableSizeFor(size + (size >>> 1) + 1);
        int sc;
        //如果不满足条件，也就是 sizeCtl < 0 ，说明有其他线程正在扩容当中，
这里也就不需要自己去扩容了，结束该方法
        while ((sc = sizeCtl) >= 0) {
            Node<K,V>[] tab = table; int n;
            //如果数组没有初始化则进行初始化，这个选项主要是为批量插入操作方
法 putAll 提供的
            if (tab == null || (n = tab.length) == 0) {
                n = (sc > c) ? sc : c;   //table 未初始化则给一个初始容量
                //初始化时将 sizeCtl 设置为 -1
                if (U.compareAndSwapInt(this, SIZECTL, sc, -1)) {
                    try {
                        if (table == tab) {
                            @SuppressWarnings("unchecked")
                            Node<K,V>[] nt = (Node<K,V>[])new Node<?,?>
[n];
                            table = nt;
                            sc = n - (n >>> 2);
                        }
                    } finally {
                      //初始化完成后 sizeCtl 用于记录当前集合的负载容量值，
也就是触发集合扩容的阈值
                        sizeCtl = sc;
                    }
                }
            }
            else if (c <= sc || n >= MAXIMUM_CAPACITY)
                break;
            //插入节点后发现链表长度达到8 个或以上，但数组长度为64 以下时触
发的扩容会进入到下面这个 else if 分支
            else if (tab == table) {
                int rs = resizeStamp(n);
                if (sc < 0) {
                    Node<K,V>[] nt;
                    if ((sc >>> RESIZE_STAMP_SHIFT) != rs || sc == rs +
 1 ||
                        sc == rs + MAX_RESIZERS || (nt = nextTable) ==
null ||
                        transferIndex <= 0)
                        break;
```

咕泡出品 必属精品

```
                    if (U.compareAndSwapInt(this, SIZECTL, sc, sc + 1))
                        //传入指定容量
                        transfer(tab, nt);
                }
                else if (U.compareAndSwapInt(this, SIZECTL, sc,
                                                (rs << RESIZE_STAMP_SHIFT)
 + 2))
                    transfer(tab, null);
            }
        }
    }
```

1.2.4 并发扩容总结

单线程新建 nextTable，新容量一般为原 table 容量的两倍。
每个线程想增/删元素时，如果访问的桶是 ForwardingNode 节点，则表明当前正处于扩容状态，协助一起扩容完成后再完成相应的数据更改操作。
扩容时将原 table 的所有桶倒序分配，每个线程每次最小分配 16 个桶，防止资源竞争导致的效率下降。单个桶内元素的迁移是加锁的，但桶范围处理分配可以多线程，在没有迁移完成所有桶之前每个线程需要重复获取迁移桶范围，直至所有桶迁移完成。
一个旧桶内的数据迁移完成但不是所有桶都迁移完成时，查询数据委托给 ForwardingNode 结点查询 nextTable 完成（这个后面看 find()分析）。
迁移过程中 sizeCtl 用于记录参与扩容线程的数量，全部迁移完成后 sizeCtl 更新为新 table 容量的 0.75 倍。

1.2.4.1 tabAt()/casTabAt()/setTabAt()

```
// 获取 Node[] 中第 i 个 Node
static final <K,V> Node<K,V> tabAt(Node<K,V>[] tab, int i)
// cas 修改 Node[] 中第 i 个 Node 的值, c 为旧值, v 为新值
static final <K,V> boolean casTabAt(Node<K,V>[] tab, int i, Node<K,V> c,
 Node<K,V> v)
// 直接修改 Node[] 中第 i 个 Node 的值, v 为新值
static final <K,V> void setTabAt(Node<K,V>[] tab, int i, Node<K,V> v)
```

1.2.4.2 sizeCtl 属性在各个阶段的作用

1.2.4.2.1 新建而未初始化时

```
int cap = ((initialCapacity >= (MAXIMUM_CAPACITY >>> 1))
                                ? MAXIMUM_CAPACITY : tableSizeFor(initial
Capacity + (initialCapacity >>> 1) + 1));
this.sizeCtl = cap;
```

作用：sizeCtl 用于记录初始容量大小，仅用于记录集合在实际创建时应该使用的大小的作用 。

咕泡出品必属精品

1.2.4.2.2 初始化过程中

```
U.compareAndSwapInt(this, SIZECTL, sc, -1)
```

作用：将 sizeCtl 值设置为 -1 表示集合正在初始化中，其他线程发现该值为 -1 时会让出 CPU 资源以便初始化操作尽快完成 。

1.2.4.2.3 初始化完成后

```
Node<K,V>[] nt = (Node<K,V>[])new Node<?,?>[n];
table = tab = nt;
sc = n - (n >>> 2);
sizeCtl = sc;
```

作用：sizeCtl 用于记录当前集合的负载容量值，也就是触发集合扩容的极限值 。

1.2.4.2.4 正在扩容时

```
//第一条扩容线程设置的某个特定基数
U.compareAndSwapInt(this, SIZECTL, sc, (rs << RESIZE_STAMP_SHIFT) + 2)
//后续线程加入扩容大军时每次加 1
U.compareAndSwapInt(this, SIZECTL, sc, sc + 1)
//线程扩容完毕退出扩容操作时每次减 1
U.compareAndSwapInt(this, SIZECTL, sc = sizeCtl, sc - 1)
```

作用：sizeCtl 用于记录当前扩容的并发线程数情况，此时 sizeCtl 的值为：((rs << RESIZE_STAMP_SHIFT) + 2) + (正在扩容的线程数)，并且该状态下 sizeCtl < 0 。

1.2.5 get 流程

```
public V get( Object key )
{
      Node<K, V>[] tab; Node<K, V> e, p; int n, eh; K ek;
      /* spread 方法能确保返回结果是正数 */
      int h = spread( key.hashCode() );
      if ( (tab = table) != null && (n = tab.length) > 0 &&
           (e = tabAt( tab, (n - 1) & h ) ) != null )
      {
      /* 如果头结点已经是要查找的 key */
            if ( (eh = e.hash) == h )
            {
                  if ( (ek = e.key) == key || (ek != null && key.equals
( ek ) ) )
                        return(e.val);
            }
            /* hash 为负数表示该 bin 在扩容中或是 treebin, 这时调用 find
方法来查找 */
            else if ( eh < 0 )
                  return( (p = e.find( h, key ) ) != null ? p.val : nul
l);
            /* 正常遍历链表, 用 equals 比较 */
            while ( (e = e.next) != null )
```

咸鱼出品必属精品

```
            {
                  if ( e.hash == h &&
                        ( (ek = e.key) == key || (ek != null && key.equa
ls( ek ) ) ) )
                         return(e.val);
            }
      }
      return(null);
}
```

1.2.6 size 计算流程

size 计算实际发生在 put，remove 改变集合元素的操作之中

- 没有竞争发生，向 baseCount 累加计数
- 有竞争发生，新建 counterCells，向其中的一个 cell 累加计数
  - counterCells 初始有两个 cell
  - 如果计数竞争比较激烈，会创建新的 cell 来累加计数

```
public int size() {
      long n = sumCount();
      return ((n < 0L) ? 0 :
            (n > (long)Integer.MAX_VALUE) ? Integer.MAX_VALUE :
            (int)n);
}
final long sumCount() {
      CounterCell[] as = counterCells; CounterCell a;
      // 将 baseCount 计数与所有 cell 计数累加
      long sum = baseCount;
      if (as != null) {
            for (int i = 0; i < as.length; ++i) {
                  if ((a = as[i]) != null)
                        sum += a.value;
            }
      }
      return sum;
}
```

- 初始化，使用 cas 来保证并发安全，懒惰初始化 table
- 树化，当 table.length < 64 时，先尝试扩容，超过 64 时，并且 bin.length > 8 时，会将链表树化，树化过程会用 synchronized 锁住链表头
- put，如果该 bin 尚未创建，只需要使用 cas 创建 bin；如果已经有了，锁住链表头进行后续 put 操作，元素添加至 bin 的尾部

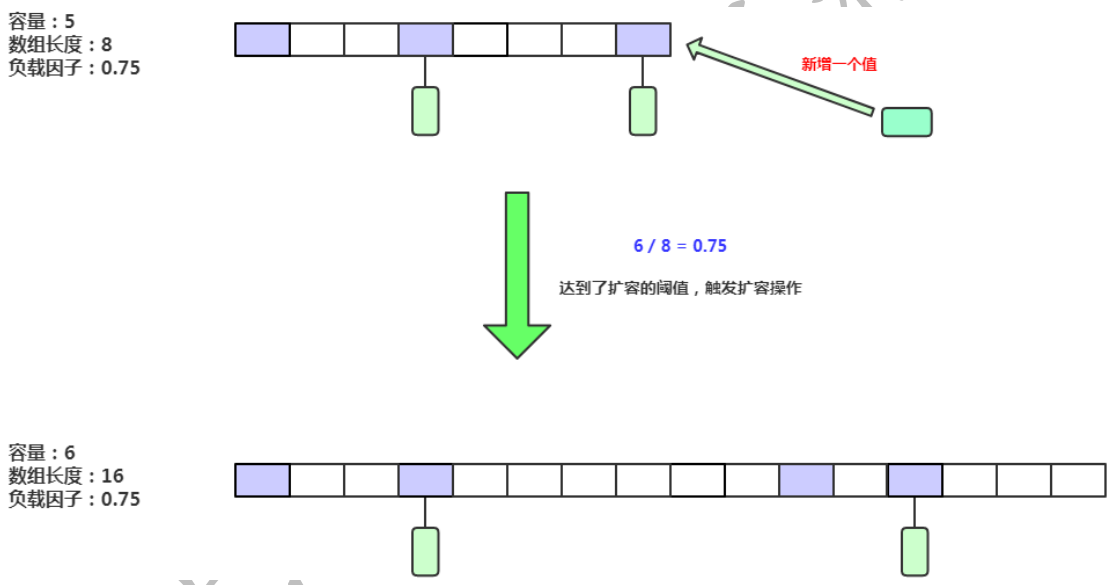
咕泡出品 必属精品

- get，无锁操作仅需要保证可见性，扩容过程中 get 操作拿到的是 ForwardingNode 它会让 get 操作在新 table 进行搜索
- 扩容，扩容时以 bin 为单位进行，需要对 bin 进行 synchronized，但这时妙的是其它竞争线程也不是无事可做，它们会帮助把其它 bin 进行扩容，扩容时平均只有 1/6 的节点会把复制到新 table 中
- size，元素个数保存在 baseCount 中，并发时的个数变动保存在 CounterCell[] 当中。最后统计数量时累加即可。

1.3 图解扩容

触发扩容的操作：

假设目前数组长度为 8，数组的元素的个数为 5。再放入一个元素就会触发扩容操作。



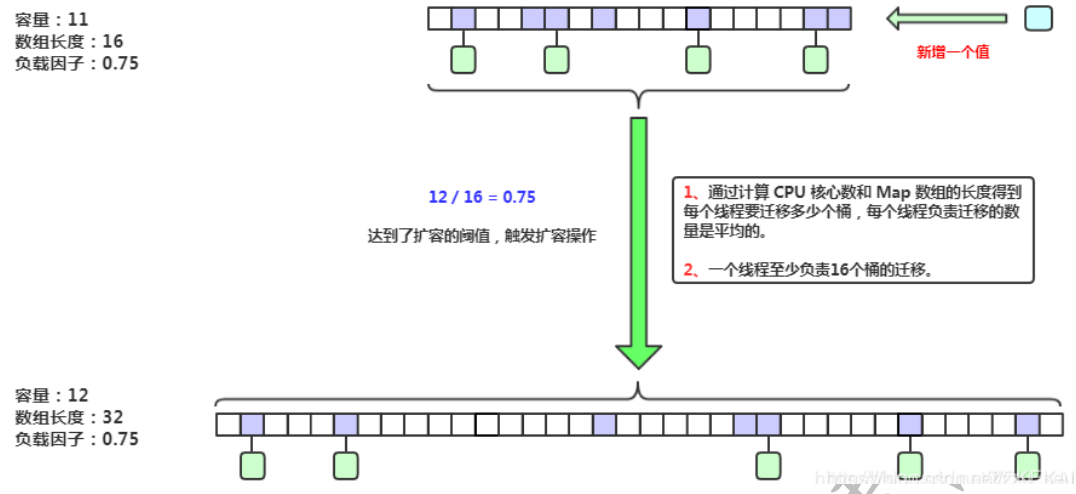
总结一下扩容条件：

(1) 元素个数达到扩容阈值。

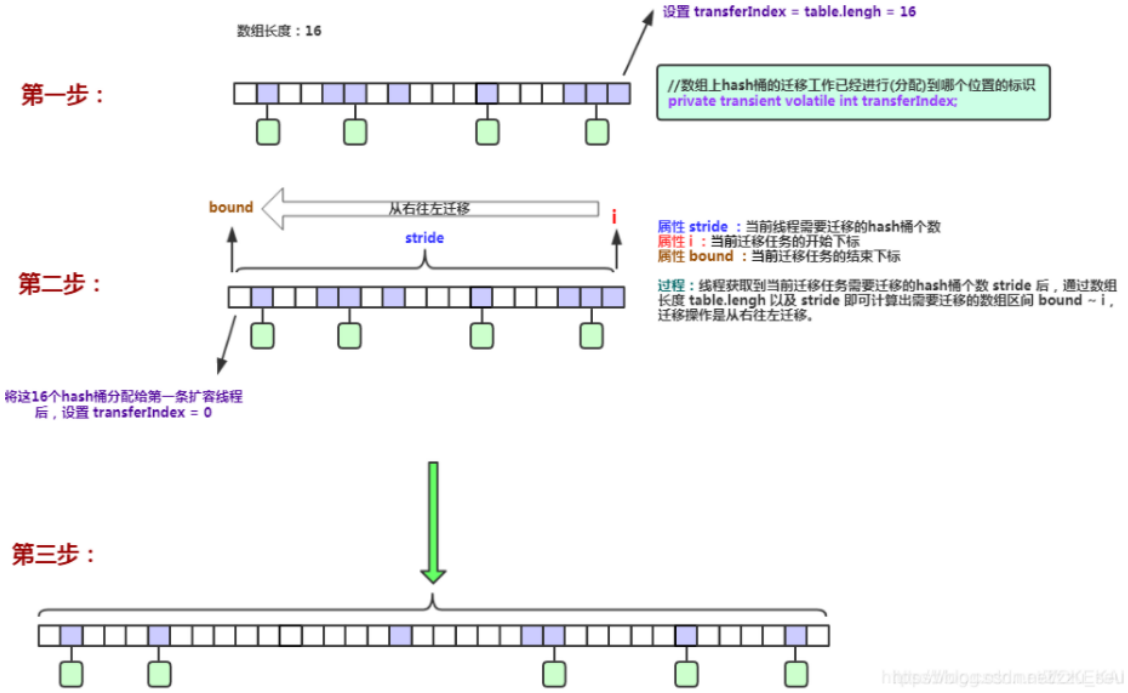
(2) 调用 putAll 方法，但目前容量不足以存放所有元素时。

(3) 某条链表长度达到 8，但数组长度却小于 64 时。

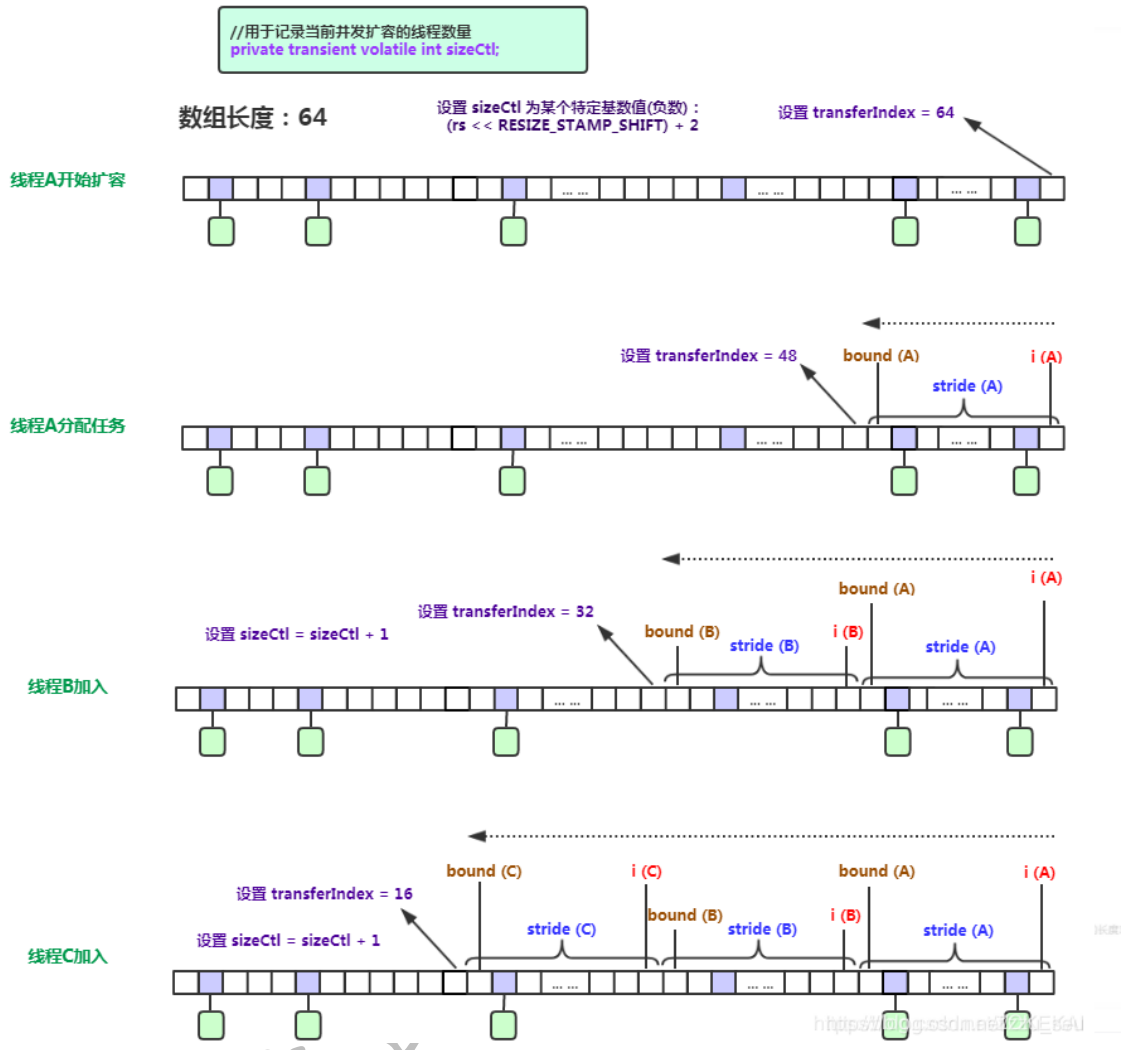
CPU 核数与迁移任务 hash 桶数量分配(步长)的关系



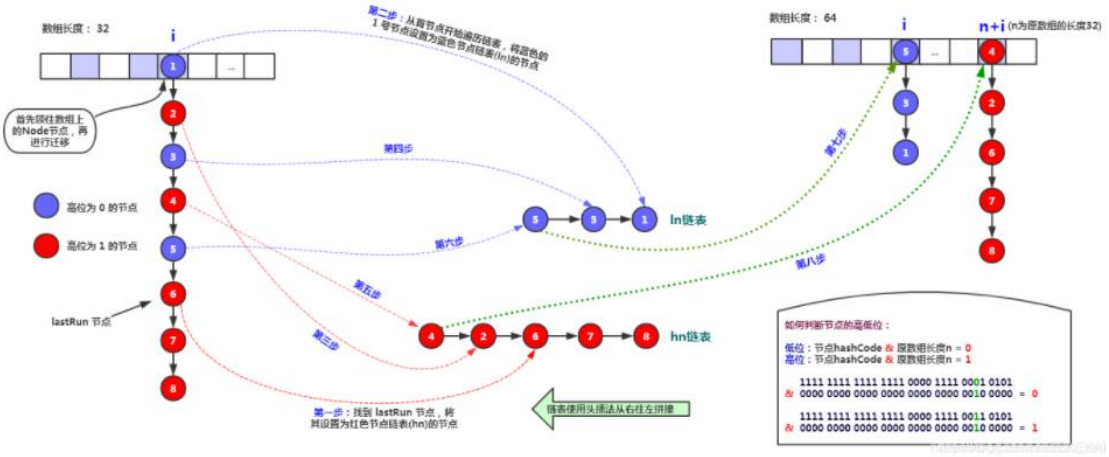
单线程下线程的任务分配与迁移操作



多线程如何分配任务？

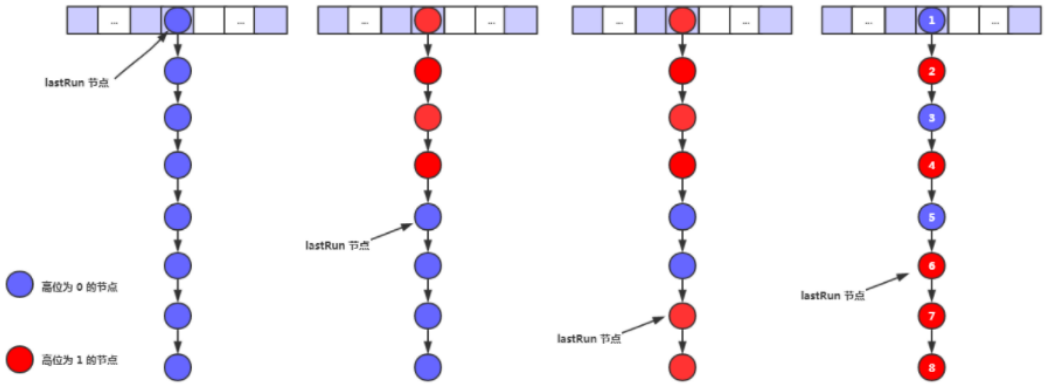

//用于记录当前并发扩容的线程数量
private transient volatile int sizeCtl;
数组长度：64
设置 sizeCtl 为某个特定基数值(负数)：
(rs << RESIZE_STAMP_SHIFT) + 2
设置 transferIndex = 64
线程A开始扩容
设置 transferIndex = 48
bound (A)
i (A)
stride (A)
线程A分配任务
设置 transferIndex = 32
设置 sizeCtl = sizeCtl + 1
bound (B)
stride (B)
i (B)
bound (A)
i (A)
stride (A)
线程B加入
设置 transferIndex = 16
设置 sizeCtl = sizeCtl + 1
bound (C)
stride (C)
i (C)
bound (B)
stride (B)
i (B)
bound (A)
i (A)
stride (A)
线程C加入

普通链表如何迁移？

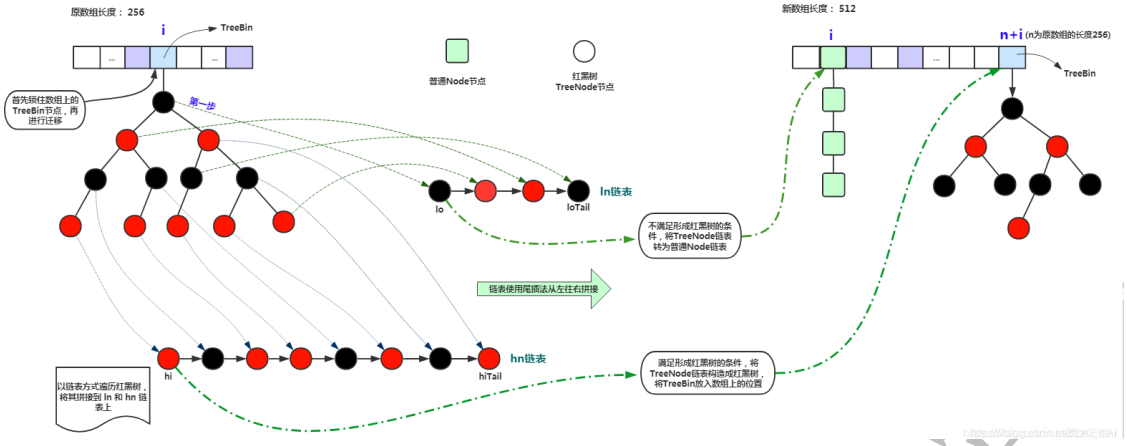


首先锁住数组上的 Node 节点，然后和 HashMap1.8 中一样，将链表拆分为高位链表和低位链表两个部分，然后复制到新的数组中。

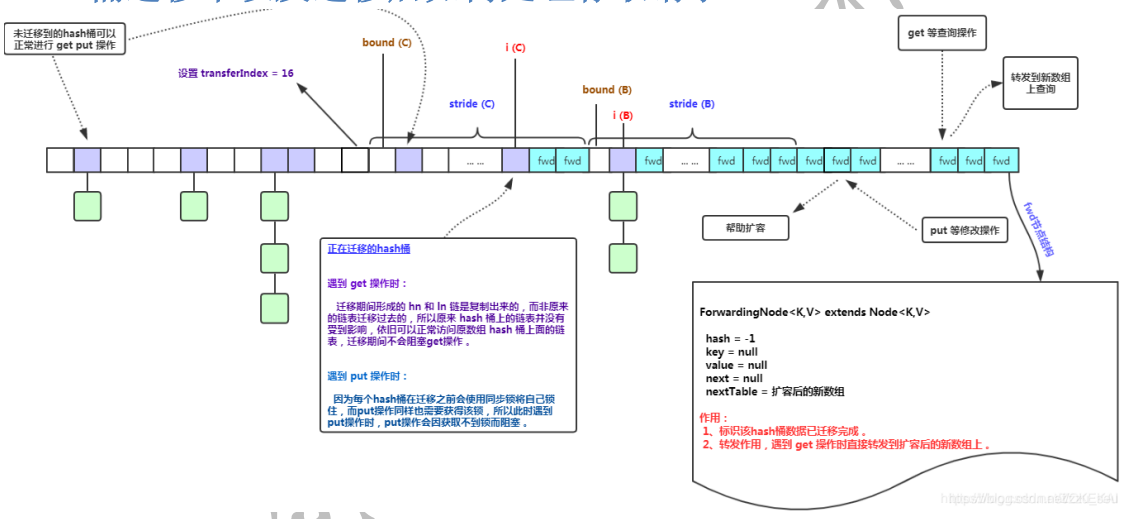
什么是 lastRun 节点？



红黑树如何迁移？



hash 桶迁移中以及迁移后如何处理存取请求？



多线程迁移任务完成后的操作

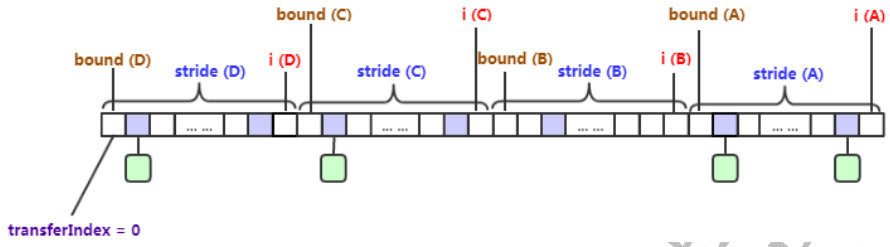
这个时候：

(1) 数组上的 hash 桶已分配完，所以 transferIndex = 0

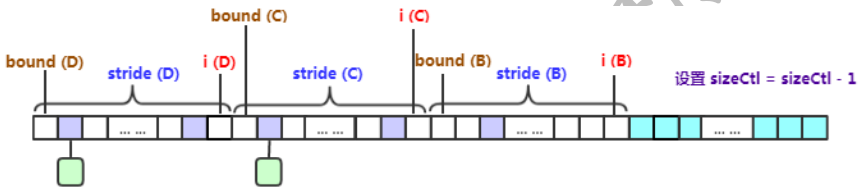
(2) 总共有4条线程在扩容，所以 sizeCtl = ((rs << RESIZE_STAMP_SHIFT) + 2) + 3

[第一条线程进行扩容时只给 sizeCtl 设置了个 基数值而没有加 1]

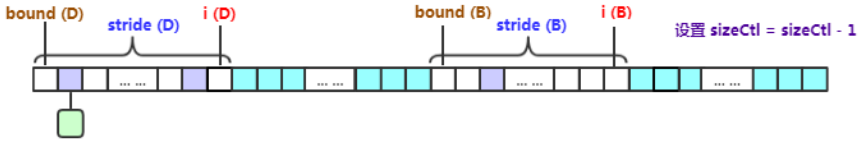
四条线程并发迁移



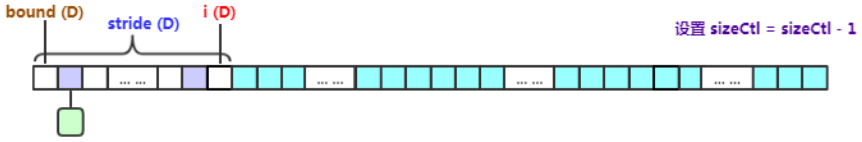
A线程迁移完成



C线程迁移完成



B线程迁移完成



D线程迁移完成

这个时候 sizeCtl 总共减了 3，所以 sizeCtl = (rs << RESIZE_STAMP_SHIFT) + 2，也就是最开始设置的基数值，根据这个可以判断所有线程都已完成迁移任务，但是作为最后退出的 D 线程，为了保险起见，在退出任务前会再次重新遍历整个 table 检查是否有遗漏的 hash 桶，有的话将其迁移到新数组。

1.4 面试题

1.4.1 JDK1.8 中的 ConcurrentHashMap 是如何保证线程安全的？

主要利用Unsafe操作+synchronized关键字。
Unsafe操作的使用仍然和JDK7中的类似，主要负责并发安全的修改对象的属性或数组某个位置的值。

synchronized主要负责在需要操作某个位置时进行加锁（该位置不为空），比如向某个位置的链表进行插入结点，向某个位置的红黑树插入结点。

JDK8中其实仍然有分段锁的思想，只不过JDK7中段数是可以控制的，而JDK8中是数组的每一个位置都有一把锁。

当向ConcurrentHashMap中put一个key,value时，

1. 首先根据key计算对应的数组下标i，如果该位置没有元素，则通过自旋的方法去向该位置赋值。
2. 如果该位置有元素，则synchronized会加锁
3. 加锁成功之后，在判断该元素的类型
   a. 如果是链表节点则进行添加节点到链表中
   b. 如果是红黑树则添加节点到红黑树
4. 添加成功后，判断是否需要进行树化
5. addCount，这个方法的意思是ConcurrentHashMap的元素个数加1，但是这个操作也是需要并发安全的，并且元素个数加1成功后，会继续判断是否要进行扩容，如果需要，则会进行扩容，所以这个方法很重要。
6. 同时一个线程在put时如果发现当前ConcurrentHashMap正在进行扩容则会去帮助扩容。

blog.csdn.net/zzu_seu

模板 2：

1. 储存 Map 数据的数组时被 volatile 关键字修饰，一旦被修改，其他线程就可见修改。因为是数组存储，所以只有改变数组内存值是才会触发 volatile 的可见性

2. 如果 put 操作时 hash 计算出的槽点内没有值，采用自旋+CAS 保证 put 一定成功，且不会覆盖其他线程 put 的值

3. 如果 put 操作时节点正在扩容，即发现槽点为转移节点，会等待扩容完成后再进行 put 操作，保证扩容时老数组不会变化

4. 对槽点进行操作时会锁住槽点，保证只有当前线程能对槽点上的链表或红黑树进行操作

5. 红黑树旋转时会锁住根节点，保证旋转时线程安全

1.4.2 JDK7 和 JDK8 中的 ConcurrentHashMap 不同点。

这两个的不同点太多了...，既包括了HashMap中的不同点，也有其他不同点，比如：

1. JDK8中没有分段锁了，而是使用synchronized来进行控制
2. JDK8中的扩容性能更高，支持多线程同时扩容，实际上JDK7中也支持多线程扩容，因为JDK7中的扩容是针对每个Segment的，所以也可能多线程扩容，但是性能没有JDK8高，因为JDK8中对于任意一个线程都可以去帮助扩容
3. JDK8中的元素个数统计的实现也不一样了，JDK8中增加了CounterCell来帮助计数，而JDK7中没有，JDK7中是put的时候每个Segment内部计数，统计的时候是遍历每个Segment对象加锁统计

https://blog.csdn.net/zzu_seu

1.4.3 扩容期间在未迁移到的 hash 桶插入数据会发生什么？

只要插入的位置扩容线程还未迁移到，就可以插入，当迁移到该插入的位置时，就会阻塞等待插入操作完成再继续迁移 。

1.4.4 正在迁移的 hash 桶遇到 get 操作会发生什么？

在扩容过程期间形成的 hn 和 ln 链 是使用的类似于复制引用的方式，也就是说 ln 和 hn 链是复制出来的，而非原来的链表迁移过去的，所以原来 hash 桶上的链表并没有受到影响，因此如果当前节点有数据，还没迁移完成，此时不影响读，能够正常进行。

如果当前链表已经迁移完成，那么头节点会被设置成 fwd 节点，此时 get 线程会帮助扩容。

1.4.5 正在迁移的 hash 桶遇到 put/remove 操作会发生什么？

如果当前链表已经迁移完成，那么头节点会被设置成 fwd 节点，此时写线程会帮助扩容，如果扩容没有完成，当前链表的头节点会被锁住，所以写线程会被阻塞，直到扩容完成。

1.4.6 如果 lastRun 节点正好在一条全部都为高位或者全部都为低位的链表上，会不会形成死循环？

在数组长度为 64 之前会导致一直扩容，但是到了 64 或者以上后就会转换为红黑树，因此不会一直死循环 。

1.4.7 扩容后 ln 和 hn 链不用经过 hash 取模运算，分别被直接放置在新数组的 i 和 n + i 的位置上，那么如何保证这种方式依旧可以用过 h & (n - 1) 正确算出 hash 桶的位置？

如果 fh & n-1 = i ，那么扩容之后的 hash 计算方法应该是 fh & 2n-1 。 因为 n 是 2 的幂次方数，所以 如果 n=16， n-1 就是 1111(二进制)， 那么 2n-1 就是 11111 (二进制) 。 其实 fh & 2n-1 和 fh & n-1 的值区别就在于多出来的那个 1 => fh & (10000) 这个就是两个 hash 的区别所在 。而 10000 就是 n 。所以说 如果 fh 的第五 bit 不

唐尧出品 必属精品

是 1 的话 fh & n = 0 => fh & 2n-1 == fh & n-1 = i 。 如果第 5 位是 1 的话 。fh & n = n => fh & 2n-1 = i+n 。

1.4.8 并发情况下，各线程中的数据可能不是最新的，那为什么 get 方法不需要加锁？

get 操作全程不需要加锁是因为 Node 的成员 val 是用 volatile 修饰的，在多线程环境下线程 A 修改结点的 val 或者新增节点的时候是对线程 B 可见的。。

1.4.9 ConcurrentHashMap 和 Hashtable 的区别?

ConcurrentHashMap 和 Hashtable 的区别主要体现在实现线程安全的方式上不同。
底层数据结构：
JDK1.7 的 ConcurrentHashMap 底层采用 分段的数组+链表 实现，JDK1.8 采用的数据结构跟 HashMap1.8 的结构一样，数组+链表/红黑二叉树。Hashtable 是采用 数组+链表 的形式。
实现线程安全的方式（重要）： ① 在 JDK1.7 的时候，ConcurrentHashMap（分段锁）对整个桶数组进行了分割分段(Segment)，每一把锁只锁容器其中一部分数据，多线程访问容器里不同数据段的数据，就不会存在锁竞争，提高并发访问率。到了 JDK1.8 的时候已经摒弃了 Segment 的概念，而是直接用 Node 数组+链表+红黑树的数据结构来实现，并发控制使用 synchronized 和 CAS 来操作。② Hashtable(同一把锁) :使用 synchronized 来保证线程安全，效率非常低下。当一个线程访问同步方法时，其他线程也访问同步方法，可能会进入阻塞或轮询状态，如使用 put 添加元素，另一个线程不能使用 put 添加元素，也不能使用 get，竞争会越来越激烈效率越低。

1.4.10 ConcurrentHashMap 和 HashMap 的相同点和不同点

相同之处：

- 都是数组 +链表+红黑树的数据结构（JDK8 之后），所以基本操作的思想一致
- 都实现了 Map 接口，继承了 AbstractMap 操作类，所以方法大都相似，可以相互切换不同之处：
- ConcurrentHashMap 是线程安全的，多线程环境下，无需加锁直接使用
- ConcurrentHashMap 多了转移节点，主要用户保证扩容时的线程安全

1.4.11 扩容过程中，读访问能否访问的到数据？怎么实现的？

可以的。当数组在扩容的时候，会对当前操作节点进行判断，如果当前节点还没有被设置成 fwd 节点，那就可以进行读写操作，如果该节点已经被处理了，那么当前线程也会加入到扩容的操作中去。

达内出品必看精品

1.4.12 为什么超过冲突超过 8 才将链表转为红黑树而不直接用红黑树?

- 默认使用链表， 链表占用的内存更小
- 正常情况下，想要达到冲突为 8 的几率非常小，如果真的发生了转为红黑树可以保证极端情况下的效率

1.4.13 ConcurrentHashMap 和 HashMap 的扩容有什么不同？

- HashMap 的扩容是创建一个新数组，将值直接放入新数组中，JDK7 采用头链接法，会出现死循环，JDK8 采用尾链接法，不会造成死循环
- ConcurrentHashMap 扩容是从数组队尾开始拷贝，拷贝槽点时会锁住槽点，拷贝完成后将槽点设置为转移节点。所以槽点拷贝完成后将新数组赋值给容器

1.4.14 ConcurrentHashMap 是如何发现当前槽点正在扩容的？

ConcurrentHashMap 新增了一个节点类型，叫做转移节点，当我们发现当前槽点是转移节点时（转移节点的 hash 值是 -1），即表示 Map 正在进行扩容.

1.4.15 描述一下 CAS 算法在 ConcurrentHashMap 中的应用

- CAS 是一种乐观锁，在执行操作时会判断内存中的值是否和准备修改前获取的值相同，如果相同，把新值赋值给对象，否则赋值失败，整个过程都是原子性操作，无线程安全问题
- ConcurrentHashMap 的 put 操作是结合自旋用到了 CAS，如果 hash 计算出的位置的槽点值为空，就采用 CAS+自旋进行赋值，如果赋值是检查值为空，就赋值，如果不为空说明有其他线程先赋值了，放弃本次操作，进入下一轮循环

咕泡出品 必属精品